ABCI (Application Blockchain Interface) 개요

개요

ABCI(Application Blockchain Interface)는 CometBFT의 핵심 구성 요소 중 하나로, **합의 엔진과 애플리케이션 로직을 분리**하는 인터페이스입니다. 이 분리를 통해 개발자들은 블록체인의 복잡한 합의 메커니즘에 대한 깊은 이해 없이도 분산 애플리케이션을 구축할 수 있습니다.

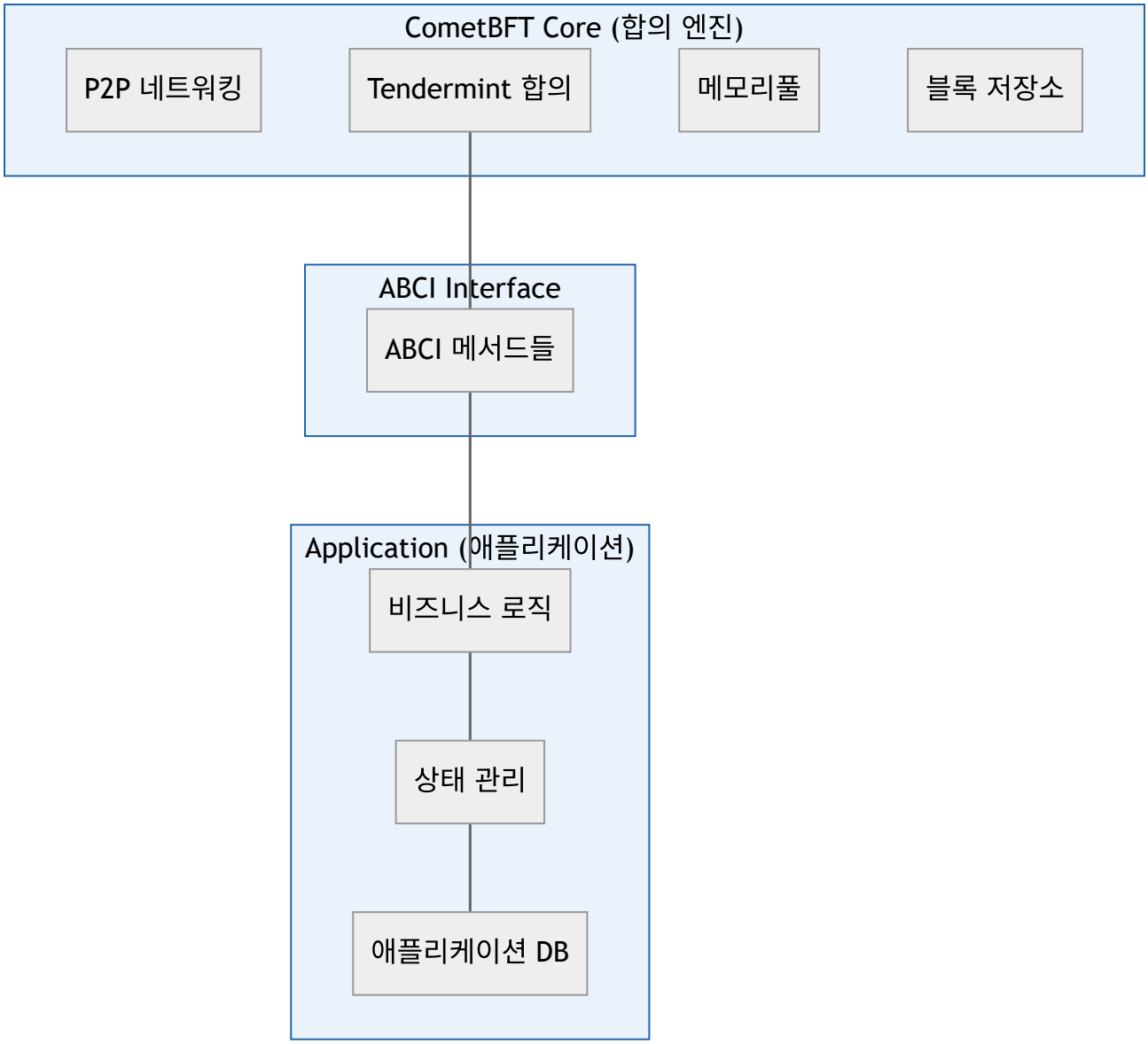
ABCI는 소켓 프로토콜을 사용하여 한 프로세스에서 실행되는 합의 엔진이 다른 프로세스에서 실행되는 애플리케이션 상태를 관리할 수 있도록 합니다. 이는 **멀티 마스터 상태 머신 복제 시스템**에서 복제 엔진(블록체인)과 상태 머신(애플리케이션) 간의 경계를 명확히 정의합니다.

ABCI의 역할과 목적

1. 관심사의 분리 (Separation of Concerns)

ABCI의 가장 중요한 목적은 **합의 로직과 애플리케이션 로직의 분리**입니다:

- **합의 엔진 (CometBFT Core)**: 네트워킹, 합의, 블록 생성 및 검증을 담당
- **애플리케이션**: 비즈니스 로직, 상태 관리, 트랜잭션 처리를 담당



2. 언어 독립성 (Language Independence)

ABCI는 프로토콜 버퍼(Protocol Buffers)를 기반으로 하여 **다양한 프로그래밍 언어**로 애플리케이션을 개발할 수 있게 합니다:

- Go, Rust, JavaScript, Python, Java 등 다양한 언어 지원
- 소켓 통신을 통한 언어 간 상호 운용성
- gRPC 서비스 정의를 통한 표준화된 통신

3. 모듈성과 재사용성

ABCI를 통해 개발자는:

- 기존 애플리케이션을 블록체인 환경에 쉽게 적용
- 다양한 합의 알고리즘과 호환되는 애플리케이션 개발
- 테스트와 디버깅이 용이한 모듈화된 구조 구현

애플리케이션-합의엔진 분리의 이점

1. 개발 복잡성 감소

```
// 개발자는 복잡한 합의 로직 대신 비즈니스 로직에만 집중
type MyApplication struct {
state  map[string]string
}

func (app *MyApplication) CheckTx(ctx context.Context, req *CheckTxRequest) (*Ch
// 트랜잭션 유효성 검사 로직만 구현
if  isValidTransaction(req.Tx) {
return &CheckTxResponse{Code: 0}, nil
}
return &CheckTxResponse{Code: 1, Log: "Invalid transaction"}, nil
}
```

2. 확장성과 성능 최적화

- **독립적인 확장**: 합의 엔진과 애플리케이션을 별도로 확장 가능

- **리소스 최적화**: 각 컴포넌트에 적합한 하드웨어 리소스 할당
- **병렬 처리**: 합의와 애플리케이션 로직의 병렬 실행

3. 유지보수성 향상

- **독립적인 업데이트**: 합의 엔진과 애플리케이션의 독립적인 버전 관리
- **테스트 용이성**: 각 컴포넌트의 독립적인 테스트 가능
- **디버깅 효율성**: 문제 영역의 명확한 분리

ABCI 인터페이스의 전체 구조

연결 유형별 분류

ABCI는 세 가지 주요 연결 유형으로 구성됩니다:

1. Info/Query 연결

```
// 애플리케이션 정보 조회
Info(ctx  context.Context, req *InfoRequest) (*InfoResponse, error)
// 상태 쿼리
Query(ctx  context.Context, req *QueryRequest) (*QueryResponse, error)
```

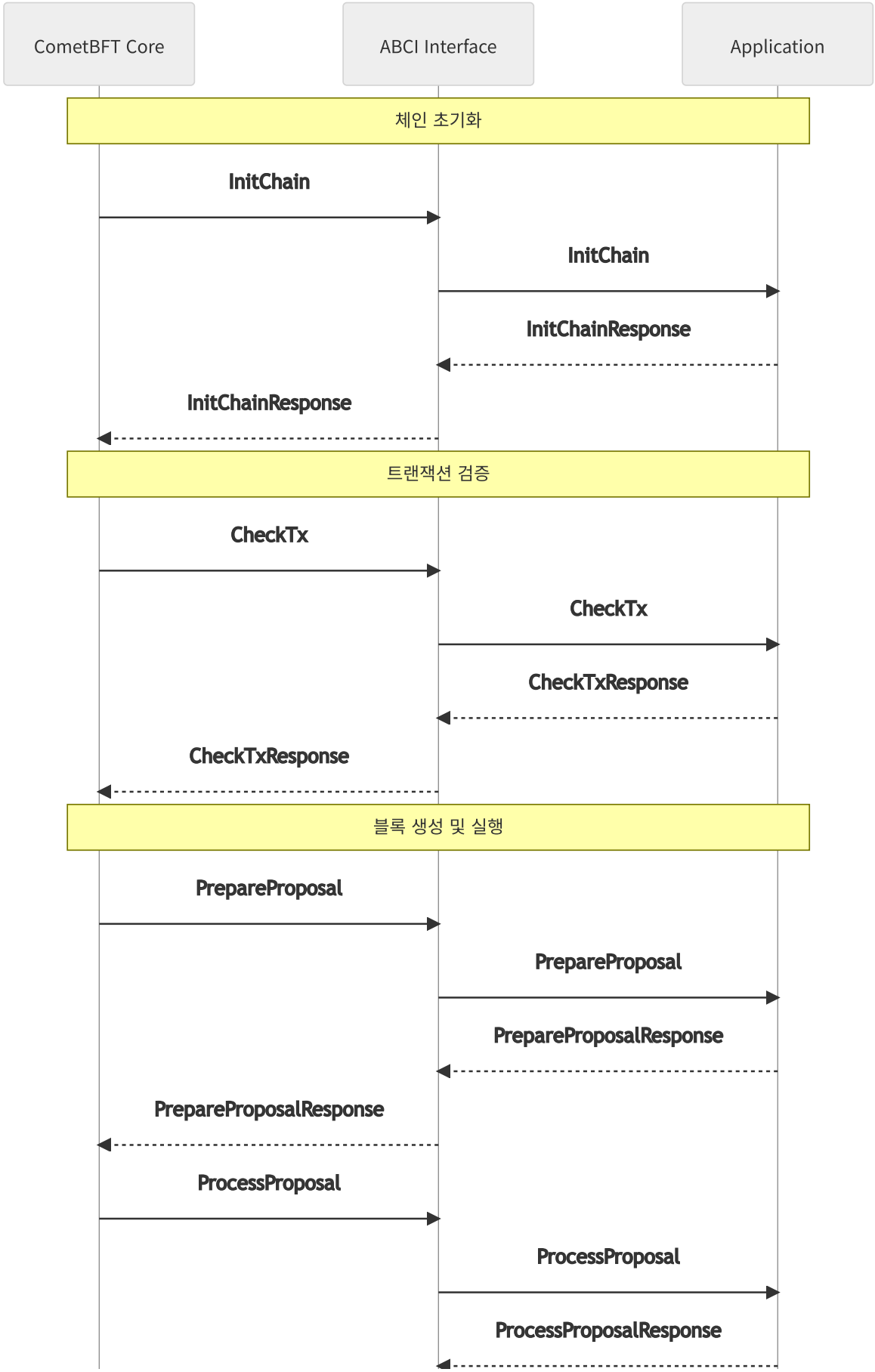
2. Mempool 연결

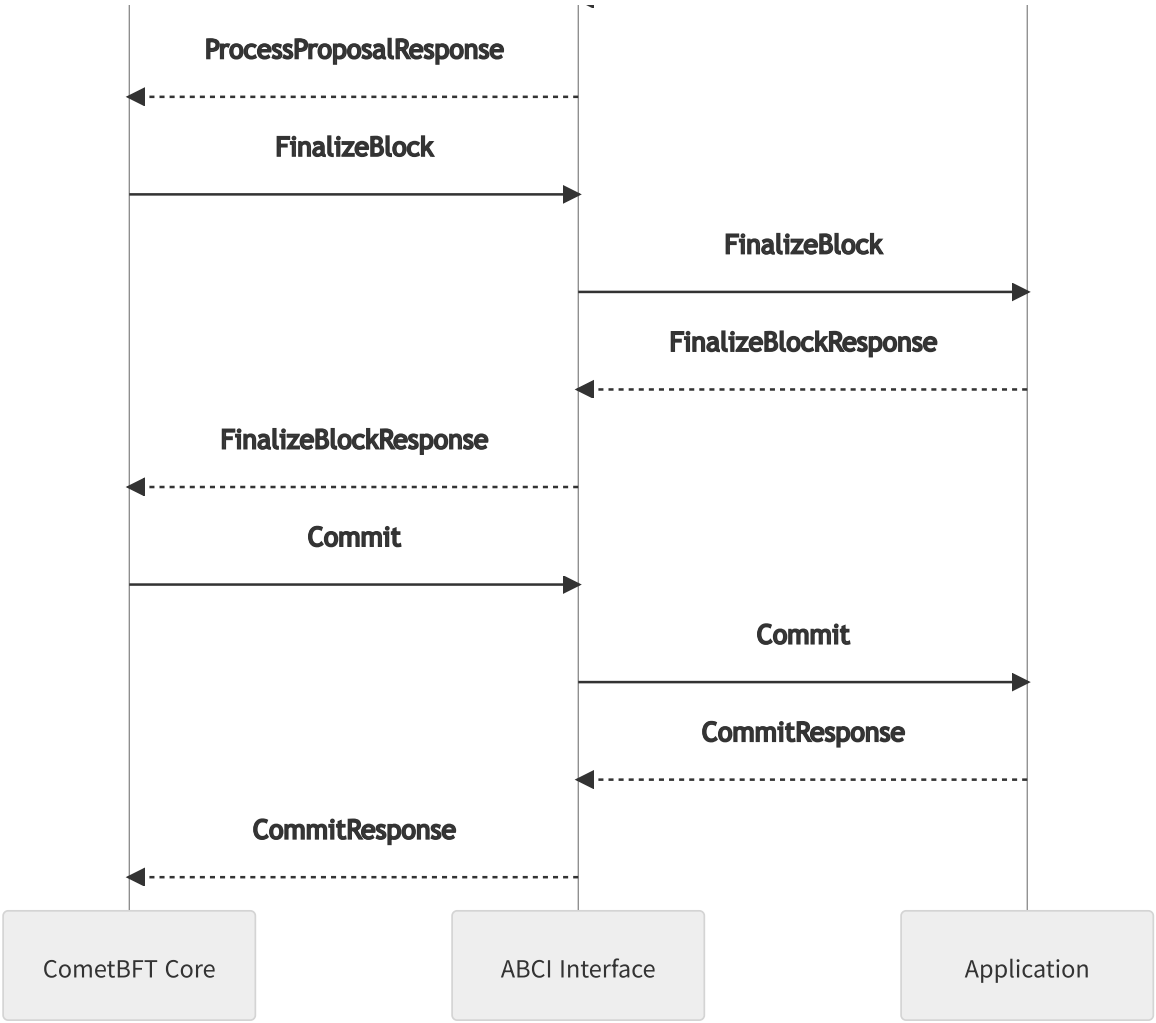
```
// 트랜잭션 유효성 검사
CheckTx(ctx  context.Context, req *CheckTxRequest) (*CheckTxResponse, error)
```

3. Consensus 연결

```
// 체인 초기화
InitChain(ctx  context.Context, req *InitChainRequest) (*InitChainResponse, erro
// 블록 제안 준비
PrepareProposal(ctx  context.Context, req *PrepareProposalRequest) (*PrepareProp
// 블록 제안 처리
ProcessProposal(ctx  context.Context, req *ProcessProposalRequest) (*ProcessProp
// 블록 최종화
FinalizeBlock(ctx  context.Context, req *FinalizeBlockRequest) (*FinalizeBlockRe
// 상태 커밋
Commit(ctx  context.Context, req *CommitRequest) (*CommitResponse, error)
```

메서드 호출 흐름


CometBFT Core
ABCI Interface
Application
체인 초기화
InitChain
InitChain
InitChainResponse
InitChainResponse
트랜잭션 검증
CheckTx
CheckTx
CheckTxResponse
CheckTxResponse
블록 생성 및 실행
PrepareProposal
PrepareProposal
PrepareProposalResponse
PrepareProposalResponse
ProcessProposal
ProcessProposal
ProcessProposalResponse



ABCI의 설계 철학

1. 결정론적 실행 (Deterministic Execution)

ABCI는 **모든 노드에서 동일한 결과**를 보장하기 위해 결정론적 실행을 요구합니다:

- 동일한 입력에 대해 항상 동일한 출력 생성
- 시간, 랜덤 값 등 비결정론적 요소 제거
- 상태 변경의 원자성 보장

2. 상태 기반 검증 (State-based Validation)

```go
// 트랜잭션 검증은 현재 상태를 기반으로 수행
func (app *MyApplication) CheckTx(ctx context.Context, req *CheckTxRequest) (*Ch
// 현재 애플리케이션 상태를 기반으로 트랜잭션 유효성 검사
currentState := app.getCurrentState()
if !isValidInCurrentState(req.Tx, currentState) {
return &CheckTxResponse{Code: 1, Log: "Invalid in current state"}, nil
}
return &CheckTxResponse{Code: 0}, nil
}
```

3. 점진적 상태 변경 (Incremental State Changes)

- **CheckTx**: 트랜잭션 유효성만 검사, 상태 변경 없음
- **FinalizeBlock**: 실제 상태 변경 수행
- **Commit**: 상태 변경 영구 저장

4. 확장 가능한 인터페이스

ABCI는 미래의 요구사항을 수용할 수 있도록 설계되었습니다:

- **Vote Extensions**: 합의 과정에서 추가 데이터 포함
- **State Sync**: 빠른 상태 동기화 지원
- **Snapshot 기능**: 상태 백업 및 복원

BaseApplication 구현

CometBFT는 개발을 쉽게 시작할 수 있도록 `BaseApplication` 을 제공합니다:

```
type BaseApplication struct{}


// 기본 구현은 모든 메서드에 대해 성공 응답 반환
func (BaseApplication) CheckTx(context.Context, *CheckTxRequest) (*CheckTxRespon
return &CheckTxResponse{Code: CodeTypeOK}, nil
}


func (BaseApplication) FinalizeBlock(_ context.Context, req *FinalizeBlockReques
txs := make([]*ExecTxResult, len(req.Txs))
for  i := range  req.Txs {
txs[i] = &ExecTxResult{Code: CodeTypeOK}
}
return &FinalizeBlockResponse{TxResults: txs}, nil
}
```

개발자는 `BaseApplication` 을 임베드하고 필요한 메서드만 오버라이드하여 애플리케이션을 구현할 수 있습니다.

다음 단계

이 개요를 통해 ABCI의 기본 개념과 설계 철학을 이해했다면, 다음 문서들을 통해 더 자세한 내용을 학습할 수 있습니다:

- ABCI 메서드 상세 분석: 각 ABCI 메서드의 구체적인 동작 방식
- 애플리케이션 생명주기: 트랜잭션 처리의 전체 흐름
- ABCI 실습 가이드: 실제 애플리케이션 구현 예제

ABCI는 CometBFT 생태계의 핵심이며, 이를 이해하는 것은 효과적인 블록체인 애플리케이션 개발의 첫 걸음입니다.